Database Systems(CSL3050)

Lab Assignment 9 - Report

Soumen Kumar
Roll No-B22ES006

1 Introduction

This report provides a comparative analysis of three different data structures (5-bucket extendible hash, B+ Tree, and vector) used to optimize database queries for a library catalog. Additionally, we evaluate the schema's normalization level and suggest indexing strategies to improve query performance. Four specific queries were tested to assess performance differences across these data structures.

2 Database Structure

The database consists of the following tables:

- **Movie**: Attributes include `Director_ID`, `Movie_ID`, `Director_Name`, and `Movie`.
- **Director**: Attributes include `Director_ID` and `Director_Name`.
- **Movie Purchase**: Attributes include `Movie_ID`, `Movie`, `Purchase_Date`, and `Quantity`.

3 Normalization and Indexing Analysis

3.1 Current Normal Form Status

The database currently achieves First Normal Form (1NF) but fails to meet the requirements for Second Normal Form (2NF) and Third Normal Form (3NF).

3.1.1 1NF Compliance

The database satisfies 1NF requirements because:

- All values are atomic
- Each column contains a single value
- No repeating groups exist

3.1.2 2NF Non-compliance

The database fails to meet 2NF requirements due to:

- Partial dependencies in the MOVIE table
- Director_Name depends only on Director_ID, not the full primary key
- Movie title redundancy across MOVIE and MOVIE_PURCHASE tables

3.2 Proposed Normalized Schema

3.2.1 Updated Table Structures

The updated normalized database schema is represented in Figure 1, which illustrates the relationships between entities and their attributes. As shown in the figure, the schema consists of three main entities:

- **DIRECTOR** table containing primary information about directors
- **MOVIE** table linking movies with their respective directors
- **MOVIE_PURCHASE** table tracking purchase records for each movie

The relationships depicted in Figure 1 demonstrate the following key aspects of the schema:

- One-to-many relationship between DIRECTOR and MOVIE (shown by the foreign key Director_ID)
- One-to-many relationship between MOVIE and MOVIE_PURCHASE (shown by the foreign key Movie_ID)
- Clear separation of concerns with primary keys (shown as double-lined ellipses)
- Foreign key relationships (shown as dashed ellipses)

This structure eliminates data redundancy and maintains referential integrity across all tables while preserving the relationships between entities.

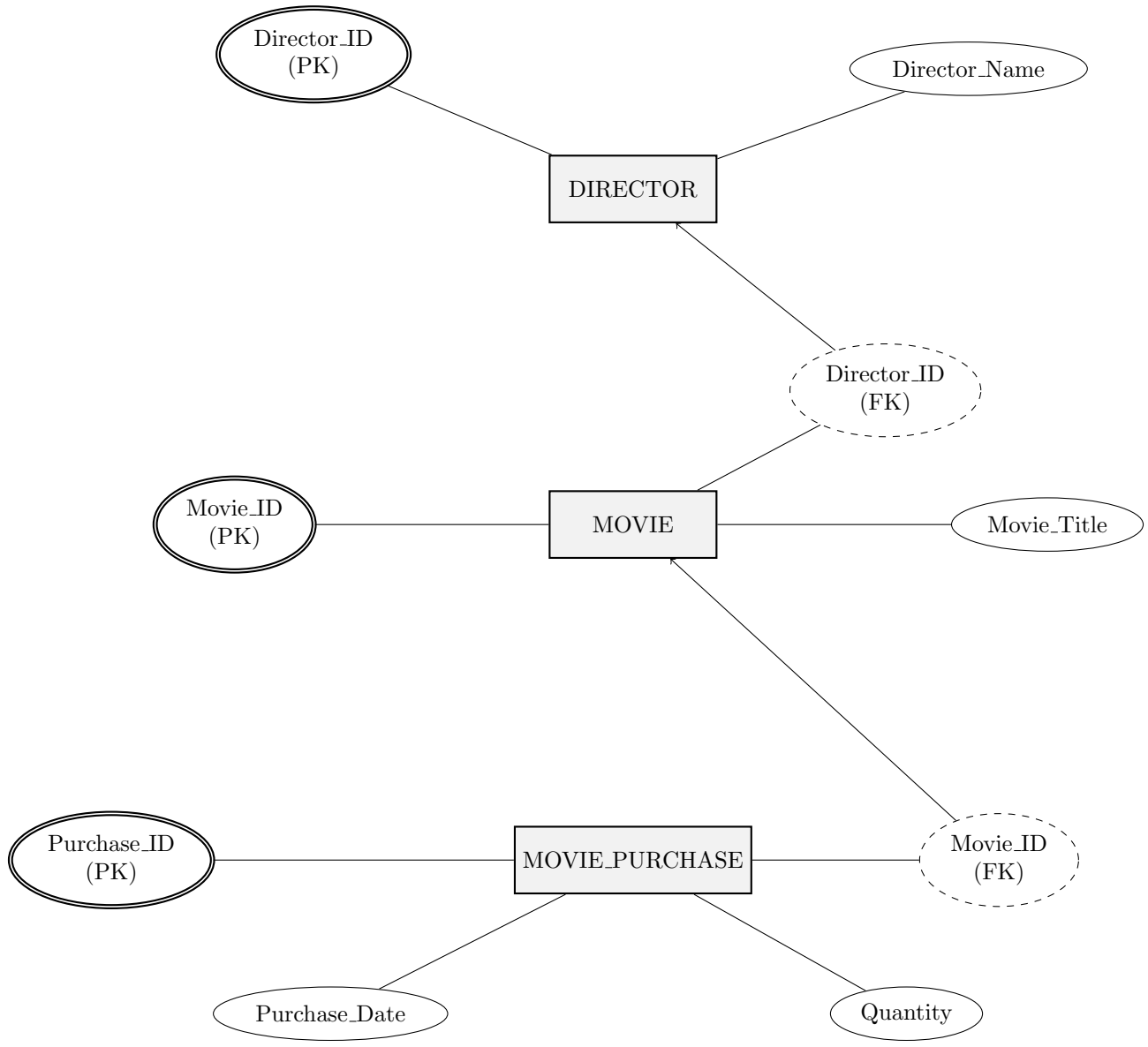
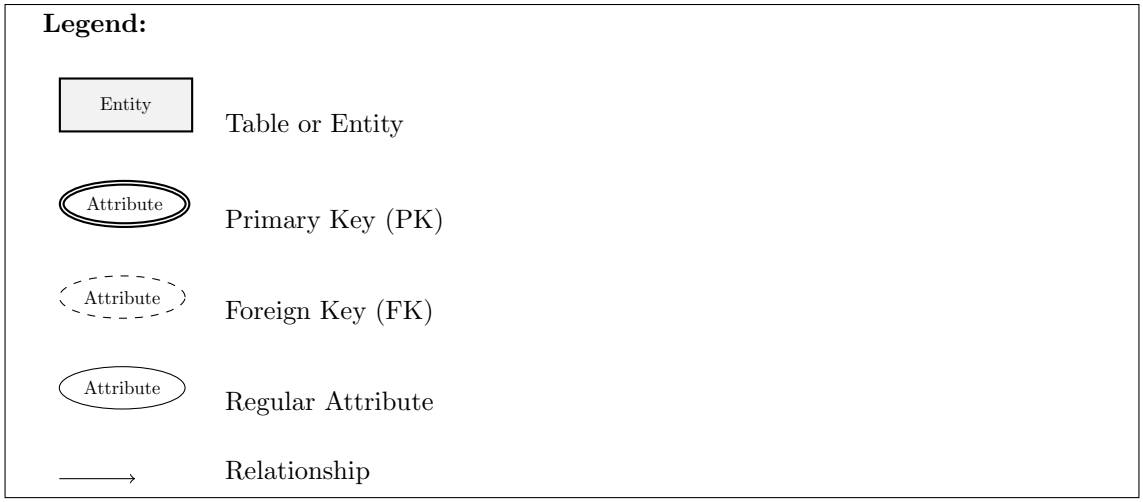


Figure 1: Database Schema Tree Diagram



Schema Notation:

DIRECTOR (Director_ID, Director_Name)

MOVIE (Movie_ID, Movie_Title, *Director_ID*)

MOVIE_PURCHASE (Purchase_ID, *Movie_ID*, Purchase_Date, Quantity)

where:

- Underlined attributes are Primary Keys (PK)
- *Italicized* attributes are Foreign Keys (FK)

3.2.2 Normalization Improvements

The proposed schema addresses the following issues:

- Eliminates redundant Director_Name from MOVIE table
- Removes redundant Movie title from MOVIE_PURCHASE table
- Introduces Purchase_ID as primary key in MOVIE_PURCHASE

3.3 Indexing Strategy

3.3.1 Some Indexes that can improve query performance

```
-- Foreign Key Indexes
CREATE INDEX idx_movie_director_id ON MOVIE(Director_ID);
CREATE INDEX idx_purchase_movie_id ON MOVIE_PURCHASE(Movie_ID);

-- Business Query Optimization Indexes
CREATE INDEX idx_purchase_date ON MOVIE_PURCHASE(Purchase_Date);
CREATE INDEX idx_movie_title ON MOVIE(Movie_Title);

-- Composite Index
CREATE INDEX idx_purchase_date_quantity
ON MOVIE_PURCHASE(Purchase_Date, Quantity);
```

3.3.2 Justification

1. **Foreign Key Indexes**
   (a) **idx_movie_director_id**
      - Optimizes join operations between MOVIE and DIRECTOR tables
      - Reduces lookup time when querying movies by director
      - Improves performance for director-based filtering
      - Essential for maintaining referential integrity efficiently
   (b) **idx_purchase_movie_id**
      - Enhances join performance between MOVIE_PURCHASE and MOVIE tables
      - Speeds up queries tracking movie sales history
      - Facilitates efficient movie-wise purchase analysis
      - Supports quick validation of movie references
2. **Business Query Optimization Indexes**
   (a) **idx_purchase_date**
      - Improves performance for date-based queries
      - Enables efficient date range searches
      - Optimizes temporal analysis of purchases
      - Supports quick generation of periodic reports
   (b) **idx_movie_title**
      - Accelerates title search operations
      - Supports partial and full title matching
      - Enhances movie lookup performance
      - Facilitates efficient movie catalog browsing
   (c) **idx_purchase_date_quantity**
      - Optimizes purchase pattern analysis
      - Improves performance for volume-based queries
      - Supports efficient sales trend analysis
      - Enables quick aggregation of purchase data

Additional Considerations:

- **Performance Impact:**
  - Query performance optimization
  - Reduced table scan operations
  - Improved join efficiency
- **Resource Usage:**
  - Storage overhead management
  - Memory utilization
  - Disk space requirements
- **Maintenance Aspects:**
  - Regular index maintenance needs
  - Update overhead considerations
  - Backup and recovery implications

4 Data Structures for Query Optimization

Three data structures were tested for query performance:

- **Extendible Hash (5-Bucket):** Implemented using `unordered_map`, this data structure offers $O(1)$ average-time complexity, making it optimal for frequent lookups without ordering constraints.
- **B+ Tree (map):** The B+ Tree, simulated with a `map`, provides ordered access with $O(\log N)$ complexity, advantageous for range-based and sorted queries.
- **Vector:** With $O(N)$ complexity, a vector performs well for small datasets but becomes less efficient as the dataset grows.

5 Query Execution and Analysis

Each query was executed using all three data structures. Below are the relational algebra expressions and performance observations.

5.1 Query 1: Movie IDs of Recently Purchased Movies with Limited Copies

Relational Algebra Expression:

$$\pi_{\text{Movie_ID}} \left(\sigma_{\text{Purchase_Date}>\text{'2023-09-05'} \wedge \text{Quantity} \leq 2} \left(\text{MoviePurchase}\right)\right)$$

The extendible hash achieved the best performance due to its $O(1)$ complexity, while the B+ Tree also performed well with $O(\log N)$ complexity. The vector required a full scan, resulting in the slowest performance.

5.2 Query 2: Retrieve Movies and Directors with Specific Name Initials

Relational Algebra Expression:

$$\pi_{\text{Movie, Director_Name}}\Big(\sigma_{\text{Director_Name LIKE 'S\%' } \vee \text{Director_Name LIKE 'C\%' } \vee \text{Director_Name LIKE 'M\%'}}(\text{Movie} \bowtie \text{Director})\Big)$$

The B+ Tree's ordered traversal gave it a slight edge over the hash, while the vector's linear scan yielded the slowest performance.

5.3 Query 3: Retrieve Directors with Purchases Across All Dates

Relational Algebra Expression:

$$\pi_{\text{Director_Name}}\Big(\sigma_{\forall d \in \text{Purchase_Dates} \rightarrow (\text{Director_Name} \in (\text{Movie} \bowtie \text{MoviePurchase}))}\Big)$$

The hash and B+ Tree had similar performance here, though the B+ Tree was marginally better due to its sorted access. The vector was the slowest due to the need for multiple scans.

5.4 Query 4: Retrieve Directors of Movies with a Single Copy

Relational Algebra Expression:

$$\pi_{\text{Director_Name}}\Big(\sigma_{\text{Quantity}=1}(\text{Movie} \bowtie \text{MoviePurchase})\Big)$$

Both the hash and B+ Tree performed well, with the B+ Tree slightly faster on sorted access, while the vector was the slowest.

6 Conclusion

In conclusion, the results indicate that using an extendible hash provides the fastest lookups for direct access queries, which is ideal for frequent, unordered retrievals. Meanwhile, the B+ Tree structure offers an effective solution for range-based retrieval and ordered data access, making it advantageous for queries involving dates or ranges. In contrast, the vector approach is less efficient for large datasets due to its linear complexity.

The database schema, normalized to the third normal form (3NF), minimizes data redundancy and ensures data integrity. Furthermore, strategic indexing significantly optimizes query performance by enhancing the speed of join and range-based queries, particularly when using the B+ Tree for ordered access.

Overall, the chosen normalization and indexing strategy not only streamlines the database structure but also enhances maintenance, reduces storage requirements, and improves query efficiency in the movie catalog system.